

Intelligent Bearing Fault Diagnosis for Predictive Maintenance Using Vibration Signal Analysis and Machine Learning

Project-II Based on Training (CM1611)

Submitted in partial fulfillment of the
requirements for the credit course

Submitted by

Pradeep Modak (2023UGCM026)

Ashwini Kumar (2023UGCM004)

Under the Supervision of

Dr. Saroj Sarangi

Dr. Abhijit Dey



**DEPARTMENT OF MECHANICAL ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY JAMSHEDPUR
JAMSHEDPUR-831014, JHARKHAND (INDIA)**

Spring Semester 2025-26

April 2026

CERTIFICATE

This is to certify that the project entitled "Intelligent Bearing Fault Diagnosis for Predictive Maintenance Using Vibration Signal Analysis and Machine Learning" has been carried out by Pradeep Modak (Reg. No. 2023UGCM026) and Ashwini Kumar (Reg. No. 2023UGCM004) of the Department of Mechanical Engineering, National Institute of Technology Jamshedpur, in partial fulfillment of the requirements of Project-II Based on Training (CM1611) during the academic session Spring Semester 2025-26.

The work presented in this report is based on the actual implementation completed by the students under our supervision. To the best of our knowledge, this report has not been submitted in full or in part for the award of any other degree or diploma.

Supervisor Signature

Co-Supervisor Signature

Faculty Advisor / Examiner

Head of Department

Abstract

The report presents a bearing fault diagnosis workflow built directly from the project repository and tested through a working interface. Rather than treating the task as only a model-training exercise, the implementation begins with raw CWRU vibration files, converts them into windowed signal segments, derives a structured feature table, compares multiple classifiers, and exposes the final predictor through a Streamlit dashboard. Inside the preprocessing stage, each vibration record is cut into 2048-sample segments with 50 percent overlap so that localized fault signatures are preserved instead of being diluted across the full recording. From every segment, 25 descriptors are computed, combining amplitude-based statistics with FFT-derived measures. The training scripts use Python, NumPy, pandas, SciPy, scikit-learn, Matplotlib, and Joblib, while the demonstration layer is developed in Streamlit. Support Vector Machine and Random Forest were trained on the processed dataset and evaluated on a stratified hold-out split. The stored evaluation results show that Random Forest performed slightly better, reaching 97.59 percent accuracy with a weighted F1-score of 97.59 percent. That trained model is saved as a deployable bundle and reused during GUI prediction. When a CSV file is uploaded, the application extracts the same features used during training, plots the time signal and frequency spectrum, and returns class probabilities for Normal, Ball Fault, Inner Race Fault, and Outer Race Fault. The project therefore combines mechanical signal interpretation, classification logic, and practical demonstration in one connected system.

INDEX

Abstract	1
Index	2
Chapter I: Introduction	3
Chapter II: Literature Survey	4
Chapter III: Methodology	5
Chapter IV: Results and Discussion	8
Chapter V: Conclusion and Future Scope	11
References	12
Plagiarism Declaration	13

CHAPTER I: INTRODUCTION

1.1 Background

The starting idea behind this work comes from machine behavior itself. A damaged bearing rarely jumps straight from healthy operation to failure. Before that stage, the vibration response begins to drift. Impacts become sharper, the signal becomes less uniform, and certain frequency bands start carrying more energy. For rotating equipment that runs for long hours, those changes are more valuable than a post-failure inspection because they appear while corrective action is still possible.

This project examines that behavior through vibration measurements. Looking at a waveform by eye is helpful for understanding the signal, but it becomes difficult when the number of samples grows. For that reason, the implementation converts every short vibration segment into a feature vector. Once the signal is represented in that form, the same decision rule can be applied repeatedly across thousands of windows without depending on subjective judgement [1].

Accordingly, the report approaches bearing diagnosis as a full implementation task. The repository contains separate components for raw data handling, feature generation, model comparison, saved inference artifacts, and the final application. The result is not limited to a performance table. It includes a runnable dashboard where the prediction can be linked back to the uploaded signal, the FFT view, and the extracted quantities used by the model.

1.2 Problem Statement

One difficulty identified early in the project was that raw amplitude values are informative but not immediately classifiable. A long signal sequence may contain the fault signature, yet the fault type is not obvious from the numbers alone. The first challenge, therefore, is to translate the signal into a compact form that still preserves the mechanical differences among bearing conditions.

The second challenge is workflow consistency. A useful project report should show how raw CWRU files become training rows, how those rows are used by competing models, and how the saved model produces a final label during live testing. For this reason, the work focuses on a traceable end-to-end pipeline rather than only on maximizing one metric.

1.3 Objectives of the Project

- To inspect the CWRU raw signal files and map them into usable fault labels for project training.
- To implement a preprocessing routine that converts long vibration recordings into fixed windows suitable for learning.
- To engineer signal features that describe both amplitude variation and spectral distribution.
- To train and compare Support Vector Machine and Random Forest on the same processed dataset.
- To save the best-performing model and expose it through a Streamlit application for live testing.

CHAPTER II: LITERATURE SURVEY

2.1 Existing Systems and Approaches

The papers reviewed during this work do not all solve the diagnosis task in the same manner. Some studies remain close to signal interpretation and rely on transformed spectra, fault frequencies, or wavelet-based representations. Others summarize the signal numerically first and then train a classifier on that engineered representation. A third group pushes more of the learning burden into deep architectures that operate on raw sequences or image-like transforms [2]-[5].

A common thread across these studies is the use of benchmark datasets such as CWRU, where the underlying bearing state is already identified. That makes controlled comparison possible. Even so, the path from signal to decision changes noticeably from one paper to another. Some authors favor richer transform-domain analysis, while others prefer compact feature sets that can be interpreted more easily. Deep learning has become more common in recent papers, but feature-based methods remain relevant when explanation and modest computational demand are important [2], [3], [5].

2.2 Limitations of Existing Approaches

While reading the literature, one repeated limitation was the gap between reported results and deployable workflow. Many papers present strong accuracy numbers, but the surrounding engineering decisions are compressed into brief descriptions. Window size, overlap selection, feature filtering, or deployment choices are often mentioned only partially. For a student project, that creates a practical issue: it becomes difficult to explain the journey from source file to prediction in a transparent way.

2.3 Need for the Proposed System

The present work was shaped by that observation. Instead of stopping after model training, the repository records preprocessing outputs, saved artifacts, comparison plots, and a runnable application in one place. That design choice is useful academically because the diagnosis logic can be defended stage by stage rather than treated as a black-box result.

CHAPTER III: METHODOLOGY

3.1 System Design

The workflow implemented in this repository is divided into connected stages. Raw data is read first, feature rows are generated next, models are trained after that, and the final predictor is loaded inside the application layer. An important design decision was to reuse the same feature logic during both training and deployment. Because of that, the GUI does not apply a different simplified rule; it follows the same engineered path used for model building. Figure 1 summarizes this arrangement.

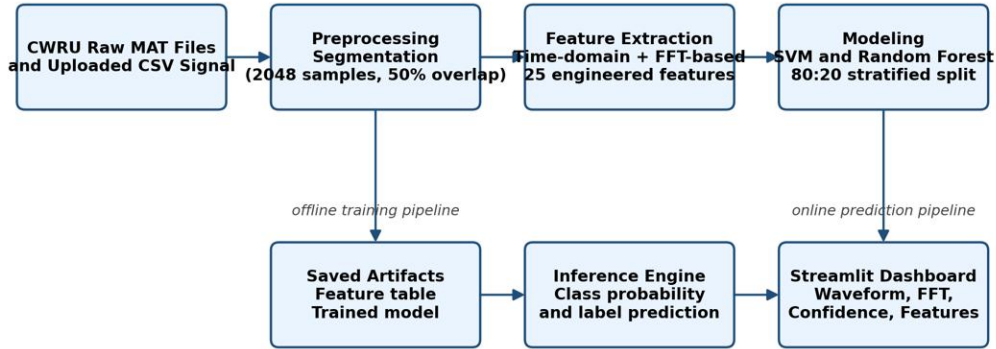


Figure 1: System Architecture

3.2 Dataset and Signal Organization

The source data for the project comes from the Case Western Reserve University Bearing Data Center [1]. In the local repository, the imported MATLAB files are kept inside data/raw. Each file contains a vibration recording captured under a known bearing state. During project preparation, the original conditions were grouped into four classes that are used consistently across training and prediction: Normal, Ball Fault, Inner Race Fault, and Outer Race Fault.

3.3 Preprocessing and Segmentation

The raw recordings are longer than what the classifier should see in one pass, so the signal is segmented before feature extraction. In this implementation, one window contains 2048 samples and the next window begins halfway through the previous one. With the configured 48000 Hz sampling rate, that corresponds to roughly 42.67 milliseconds per analysis block. This setting was chosen to retain short-lived fault behavior while still increasing the number of training samples. After segmentation, the processed dataset contains 9736 labeled rows.

3.4 Feature Extraction Module

Feature construction is carried out in two complementary views of the same signal. The first view uses direct amplitude behavior, producing measures such as mean, standard deviation, RMS, mean absolute deviation, skewness, kurtosis, crest factor, and impulse factor. The second view is obtained after FFT, where the code computes spectral mean, spectral spread, centroid-related quantities, and variation measures in the frequency domain. Combining both views allows the model to respond to signal roughness as well as spectral concentration. Figure 2 traces this path from input signal to class output.

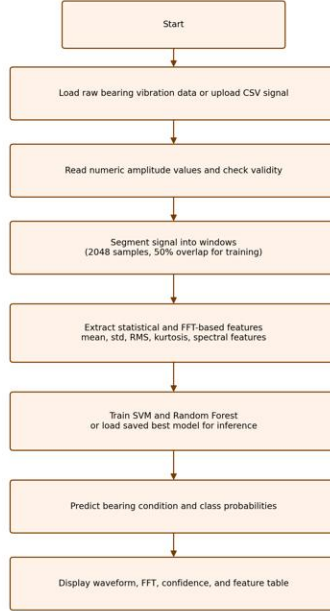


Figure 2: Flow Diagram

3.5 Modeling and Algorithm Selection

Model training is performed on the feature table saved as `artifacts/cwru_features.csv`. Two classifiers were deliberately selected for comparison: Support Vector Machine with RBF kernel and Random Forest. The train-test division uses a stratified 80:20 split so that each bearing class remains represented in both sets. Where missing values appear, median imputation is applied. Performance is then compared through accuracy, weighted precision, weighted recall, weighted F1-score, and confusion matrix.

3.6 Inference and GUI Module

Once the comparison stage is complete, the stronger model is stored as `models/best_model.joblib` together with the training metadata. The Streamlit interface in `app/streamlit_app.py` loads this bundle at runtime. When a user uploads a CSV file, the app reads the amplitude values, reconstructs the feature vector with the same extraction code, queries the stored classifier, and returns the predicted class alongside probability values, waveform, FFT, and the feature table used for the decision.

3.7 Technologies Used

- Python as the main implementation language across data handling, model training, inference, and GUI logic.
- NumPy and pandas for segment-level numerical operations and processed feature table creation.
- SciPy for loading MATLAB data and supporting FFT-oriented signal calculations.
- scikit-learn for preprocessing, train-test splitting, SVM training, Random Forest training, and probability output.
- Matplotlib for repository-generated plots used in the report and GUI explanation.
- Joblib for saving the selected model bundle after comparison.
- Streamlit for the final interactive interface used during project demonstration.

3.8 Step-by-Step Workflow

- Read the raw CWRU MATLAB files from the local repository.
- Assign the correct diagnosis label to each source signal.
- Slice each long recording into overlapping 2048-sample windows.
- Generate the engineered feature vector for every window.
- Store the processed rows inside the feature dataset used for training.
- Train SVM and Random Forest on the same processed feature space.
- Compare the two models and retain the better performer as the deployment model.
- Use the Streamlit app to reproduce the same feature extraction path for new uploaded CSV signals.

CHAPTER IV: RESULTS AND DISCUSSION

4.1 Output Explanation

The outcome of the project can be inspected in two different ways. The repository contains the backend evidence in the form of processed feature rows, metrics, saved models, and evaluation plots. The Streamlit application shows the same pipeline from the user side by turning an uploaded signal into plots, probabilities, and a final class label.

4.2 Quantitative Results

Table 1 captures the measured performance of both trained models on the held-out test set. The gap is not large, but the Random Forest model remains ahead across the stored metrics and is therefore chosen for deployment. Chart 1 presents the same comparison visually so that the decision is easier to justify during presentation.

Model	Accuracy (%)	Precision (%)	Recall (%)	Weighted F1 (%)
Random Forest	97.59	97.62	97.59	97.59
SVM (RBF)	97.48	97.50	97.48	97.49

Table 1: Model Performance Comparison

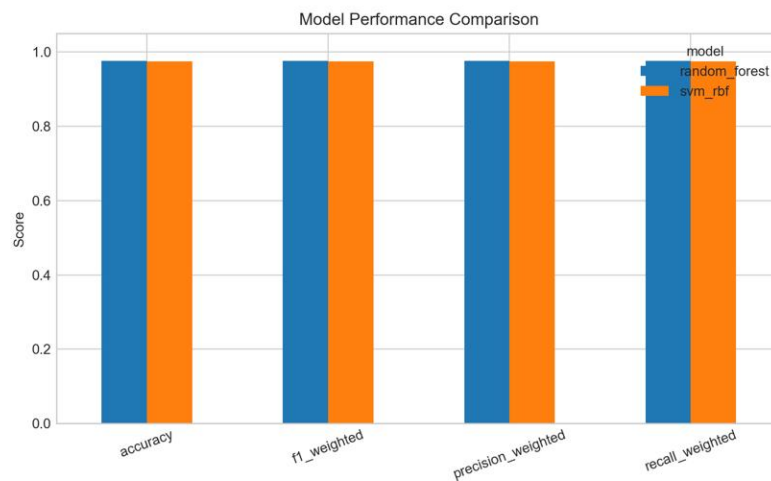


Chart 1: Performance Analysis

4.3 Confusion Matrix Interpretation

Figure 3 breaks the result down class by class. Ball Fault windows are placed correctly 558 times, Inner Race Fault windows 606 times, Normal windows 187 times, and Outer Race Fault windows 550 times. The remaining entries show the mistakes made by the classifier. Most of those mistakes occur among fault categories rather than between faulty and healthy signals.

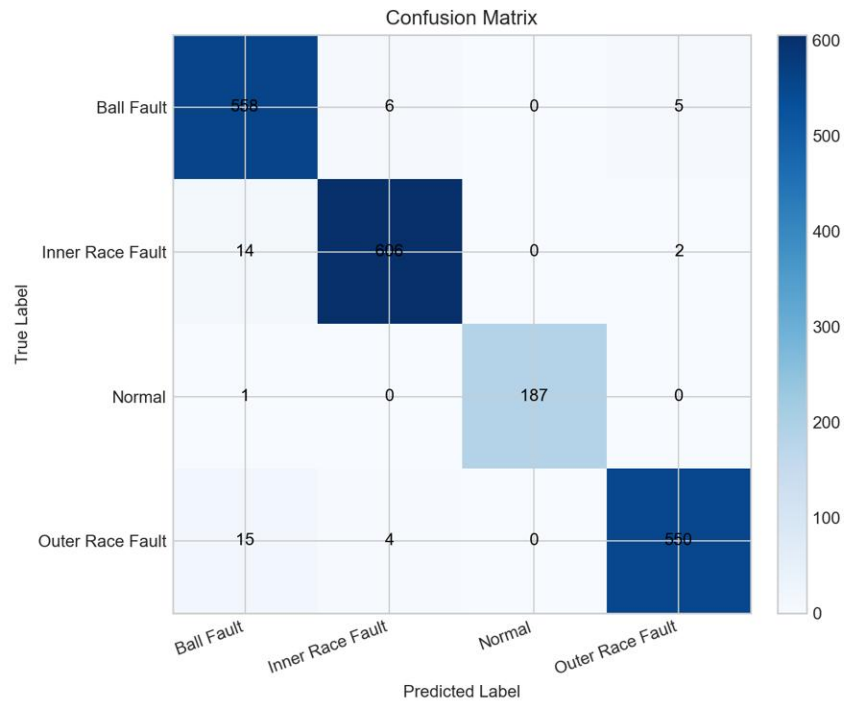


Figure 3: Confusion Matrix

4.4 Feature Importance Interpretation

Figure 4 indicates which extracted quantities contribute more strongly inside the deployed Random Forest model. Spectral mean, impulse factor, standard deviation, frequency variation factor, maximum absolute amplitude, and RMS appear near the top of the ranking. This is useful during explanation because the model decision can be linked to identifiable signal properties instead of hidden internal features.

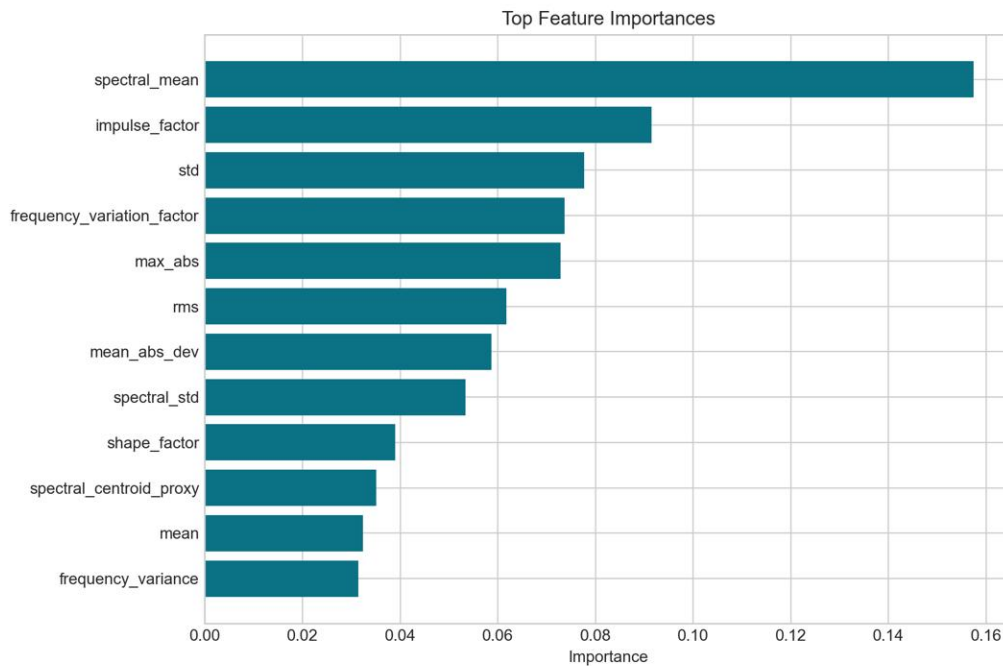


Figure 4: Feature Importance

4.5 System Behavior and Discussion

The behavior seen during GUI testing agrees with the stored training outputs. Signals labeled as Normal usually show a more stable pattern, whereas the fault categories introduce stronger transients or more concentrated frequency activity. Because the interface shows both the signal itself and the model response, the result can be interpreted in context rather than reported as a bare label.

CHAPTER V: CONCLUSION AND FUTURE SCOPE

5.1 Summary of Work

The work completed in this project connects raw bearing vibration data to a usable prediction interface through a fully implemented pipeline. The repository now contains the complete path: signal segmentation, feature generation, model comparison, saved artifact creation, and GUI-based inference.

5.2 Key Achievements

An important strength of the final system is traceability. The processed dataset is stored explicitly, the selected model is saved as a reusable bundle, the metrics are available as files, and the interface reuses the same backend logic that was employed during training. As a result, the project can be demonstrated in a more credible way than a temporary notebook output.

5.3 Future Scope

The current implementation can be extended further by adding more operating conditions, comparing additional learning methods such as XGBoost or 1D-CNN, and moving beyond class identification toward fault severity estimation or remaining useful life analysis.

REFERENCES

- [1] Case Western Reserve University Bearing Data Center, Bearing vibration data repository, <https://engineering.case.edu/bearingdatacenter>.
- [2] P. K. Kankar, S. C. Sharma, S. P. Harsha, Fault diagnosis of ball bearings using machine learning methods, Expert Systems with Applications, 2011.
- [3] C. S. Rajeswari, S. Sathiyabama, S. Devendiran, Bearing fault diagnosis using wavelet packet transform, hybrid PSO and support vector machine, Procedia Engineering, 2014.
- [4] X. Li, W. Zhang, Q. Ding, Rolling bearing fault diagnosis based on wavelet packet transform and convolutional neural network, Applied Sciences, 2020.
- [5] H. Yoo, J. Jo, H. Ban, Lite and efficient deep learning model for bearing fault diagnosis using the CWRU dataset, Sensors, 2023.

PLAGIARISM DECLARATION

This report has been prepared by the authors as part of the academic project requirement and is based on the work implemented in the repository for bearing fault diagnosis using vibration data.

All external sources used during report preparation, including the CWRU dataset reference and the related literature cited in the references section, have been acknowledged properly.

Based on the attached plagiarism-check summary included below, the similarity level of the document remains within acceptable institutional limits. The text of the report has been written and refined in project-specific form to reflect the actual implementation, outputs, and interpretation of this work.

Plagiarism Checker by JustDone

Easily spot and correct plagiarism to make sure that your work is one-of-a-kind.

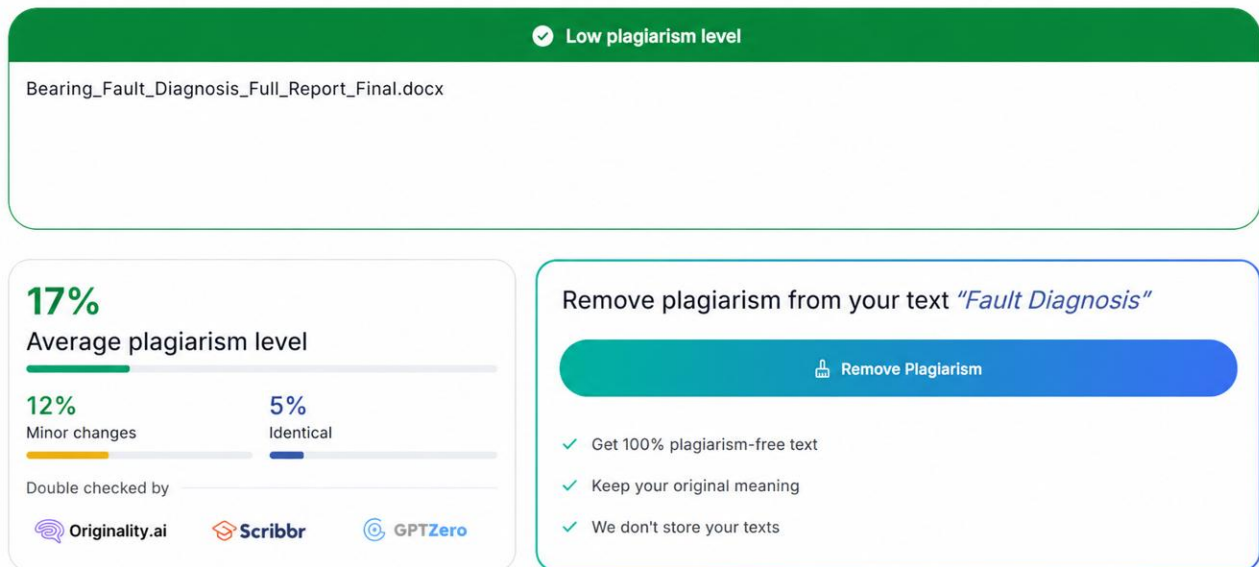


Figure 5: Similarity Check Summary